Experiment 9: Fashion MNIST Classification using Convolutional Neural Networks

Department of Electrical Engineering
Indian Institute of Technology Kharagpur
Algorithms, AI and ML Laboratory (EE22202)
Spring, 2025-26

1 Convolutional Neural Networks for Image Classification

A Convolutional Neural Network (CNN) is a specialised deep learning architecture designed to process data with a spatial grid structure, such as images. Unlike a fully connected network that flattens the input and loses all spatial relationships, a CNN operates directly on the image tensor and learns to detect local patterns like edges, textures, and shapes that are shared across different positions in the image. A CNN for classification or regression tasks is composed of three main types of layers: convolutional layers, pooling layers, and fully connected layers.

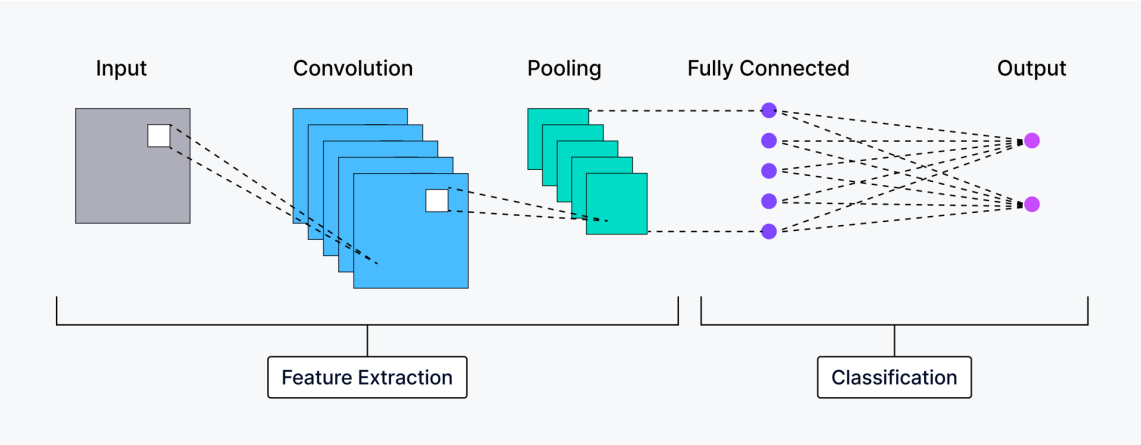

Figure 1: Basic building blocks of a CNN: a convolutional layer extracts local feature maps, max pooling spatially downsamples them, and fully connected layers perform the final classification from the flattened features.

1.1 Convolutional Layer

A convolutional layer applies a bank of learnable filters to the input feature map. Each filter W of size $k \times k$ slides across the entire input with a given stride s and padding p, computing an element-wise dot product at each position to produce a single output feature map. For a single-channel input X, the output at position (i, j) is:

$$Y(i,j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X(i \cdot s + m,\ j \cdot s + n) \cdot W(m,n) + b \tag{1}$$

where b is a learnable bias. For a multi-channel input with C_{in} channels, each filter has shape $C_{\text{in}} \times k \times k$ and the summation extends over all input channels. With C_{out} such filters, the layer

produces C_{out} output feature maps, collectively called the output volume. The output spatial size is:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1 \tag{2}$$

The total number of learnable parameters in one convolutional layer is:

$$\text{Params}_{\text{conv}} = (k \times k \times C_{\text{in}} + 1) \times C_{\text{out}} \tag{3}$$

where the +1 accounts for the bias of each filter. A non-linear activation function (for example, ReLU) is applied element-wise to the output immediately after the convolution.

1.2 Max Pooling Layer

A max pooling layer reduces the spatial dimensions of the feature map by partitioning it into non-overlapping $k \times k$ windows and retaining only the maximum value in each window:

$$Y(i,j) = \max_{0 \le m < k,\ 0 \le n < k} X(i \cdot s + m,\ j \cdot s + n) \tag{4}$$

With a 2×2 pool and stride 2, each spatial dimension is halved.

1.3 Fully Connected Layer

After the convolutional and pooling layers have extracted a hierarchy of spatial features, the resulting feature volume is flattened into a 1D vector and passed through one or more fully connected (FC) layers. FC layers may be used for classification and regression tasks. Every neuron in an FC layer is connected to every element of the previous layer:

$$\mathbf{y} = \text{ReLU}(W\mathbf{x} + \mathbf{b}) \tag{5}$$

where W is a weight matrix of shape (out, in) and $\mathbf{b}$ is a bias vector. FC layers combine the locally detected features into global representations suitable for classification. The final FC layer produces one logit per class, and a softmax function converts these logits to class probabilities:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}, \quad i = 1, 2, \ldots, C$$

where z_i is the logit corresponding to class i.

1.4 Loss Function

The network is trained by minimising the cross-entropy loss between the predicted probability vector $\hat{y}$ and the ground-truth vector y:

$$L(y, \hat{y}) = -\sum_{i=1}^{C} y_i \cdot \log(\hat{y}_i) \tag{6}$$

1.5 Dropout

Dropout is a regularisation technique used to reduce overfitting in neural networks. During training, it randomly sets the output of each neuron to zero with probability p, effectively preventing neurons from relying too heavily on specific other neurons. Mathematically, dropout can be represented as:

$$\tilde{x} = x \times m \tag{7}$$

where x is the input activation, $m \sim \text{Bernoulli}(1-p)$ is a random binary mask, and $\tilde{x}$ is the output after applying dropout. The mask m takes the value 0 with probability p and 1 with probability $1-p$, dropping a fraction p of the neurons at each training iteration.

To ensure that the expected magnitude of activations is consistent between training and inference, a scaling factor must be applied. In the commonly used *inverted dropout* convention (used by PyTorch), the surviving activations are scaled up by $\frac{1}{1-p}$ during training, so no adjustment is needed at inference. Alternatively, in *vanilla dropout*, no scaling is applied during training, and all activations are instead multiplied by $(1-p)$ at inference time.

1.6 Batch Normalisation

Batch Normalisation (BN) normalises the output of a layer across the current mini-batch before the activation function is applied. For a mini-batch of activations $\{x_1, \ldots, x_B\}$, the normalised output is:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \tag{8}$$

where $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the mean and variance of the mini-batch, and ϵ is a small constant for numerical stability. The normalised value is then scaled and shifted by learnable parameters γ and β:

$$y_i = \gamma \hat{x}_i + \beta \tag{9}$$

This ensures the layer can still represent the identity transform if needed. At inference time, $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are replaced by running estimates accumulated during training.

2 Experiments

2.1 Importing Libraries

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
```

Listing 1: Code to import essential libraries

2.2 Download and Preparation of the Dataset. Creating the Dataloaders.

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
train_dataset = datasets.FashionMNIST(
    'F_MNIST_data', download=True, train=True, transform=transform)
test_dataset  = datasets.FashionMNIST(
    'F_MNIST_data', download=True, train=False, transform=transform)

indices = list(range(len(train_dataset)))
np.random.shuffle(indices)
split = int(0.2 * len(train_dataset))
val_ids, train_ids = indices[:split], indices[split:]

```

```python
train_sampler = torch.utils.data.SubsetRandomSampler(train_ids)
val_sampler   = torch.utils.data.SubsetRandomSampler(val_ids)

BATCH_SIZE = 128
train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size=BATCH_SIZE, sampler=train_sampler)
val_loader   = torch.utils.data.DataLoader(
    train_dataset, batch_size=BATCH_SIZE, sampler=val_sampler)
test_loader  = torch.utils.data.DataLoader(
    test_dataset, batch_size=BATCH_SIZE, shuffle=False)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

Listing 2: Code to download the dataset and prepare dataloaders

2.3 Visualisation of the Dataset

```python
print("Length of train data is " + str(len(train_sampler)))
print("Length of test data is "  + str(len(test_dataset)))
print("Length of validation data is " + str(len(val_sampler)))

image, label = next(iter(train_loader))
print(image[0].shape, label.shape)  # (1, 28, 28)

desc = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
        'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle Boot']
print(desc[label[0].item()])
plt.imshow(image[0].numpy().squeeze(), cmap='gray')
```

Listing 3: Code to visualise the dataset

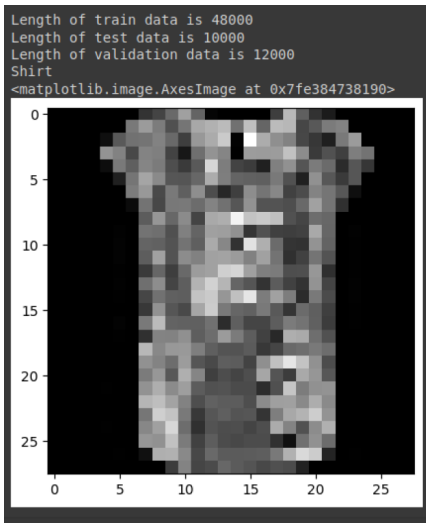

Figure 2: The first image and label of the training data.

2.4 Base Model

$$\text{Input}_{1\times28\times28} \to \underbrace{\text{Conv}(64, 2\times2) \to \text{ReLU} \to \text{Pool}(2\times2)}_{\text{Block 1}} \to \underbrace{\text{Conv}(64, 2\times2) \to \text{ReLU} \to \text{Pool}(2\times2)}_{\text{Block 2}}$$

$$\to \underbrace{\text{Conv}(64, 2\times2) \to \text{ReLU}}_{\text{Block 3}} \to \text{Flatten} \to \underbrace{\text{FC}(64) \to \text{ReLU} \to \text{FC}(10)}_{\text{10-class classification}}$$

3 Assignments to Solve and Report

For each experiment below, report (a) the total number of **trainable parameters**, (b) the **training loss and validation loss** for 80-20 train-validation split, (c) training, validation and test **accuracies**, and (d) **activation maps** of every convolutional layer for one image after every 10 epochs.

1. **Baseline**: Train `BaseCNN()` with default parameters: kernel size $k = 2$, stride $s = 1$, `num_blocks`= 3, filters= 64, `fc_hidden`= 64, no dropout, and no batch normalisation. Use the Adam optimiser with learning rate 0.001, batch size 128, and train for 50 epochs.

2. **Kernel Size and Stride Ablation**: Retrain `BaseCNN` with kernel size $k = 5$ and stride $s = 2$ in every convolutional layer, keeping all other parameters unchanged from `BaseCNN`.

3. **Depth Ablation (shallower)**: Retrain `BaseCNN` with `num_blocks`= 2, keeping all other parameters unchanged from `BaseCNN`.

4. **Dropout ($p = 0.25$)**: Retrain `BaseCNN` with dropout probability $p = 0.25$ applied after the hidden FC layer, keeping all other parameters unchanged from `BaseCNN`.

5. **Batch Normalisation**: Retrain the best-performing model by adding `nn.BatchNorm2d` after each convolutional layer.

Table 1: Summary of all experiments. Fixed hyperparameters across all runs: Adam optimiser, lr= 0.001, batch size= 128, 50 epochs, filters= 64, `fc_hidden`= 64. Entries in **bold** indicate the parameter changed from the baseline.

Exp No.	Kernel	Stride	Depth	Dropout	BN
1	2	1	3	0.00	No
2	**5**	**2**	3	0.00	No
3	2	1	**2**	0.00	No
4	2	1	3	**0.25**	No
5	best-model	best-model	best-model	best-model	**Yes**